

Manual Operativo de Datos

Proyecto: Pipeline de Generación y Publicación de Contenido con IA — Entrega Final

1. Esquema de bases de Airtable

El sistema usa 3 tablas dentro de la base "Pipeline de Contenido", más una tabla externa de solo lectura para RAG.

Tabla 1: "Manual de Marca" (fuente RAG — solo lectura desde Make)

Campo	Tipo	Descripción
Categoría	Single Select	Clasifica el fragmento (Tono de Voz, Servicios, Precios, FAQs)
Contenido	Long Text	El texto validado que se usa como contexto para la IA
Estado de Verificación	Single Select	Filtra solo los fragmentos aprobados (vista "Verificada")

Relación: se vincula lógicamente con "Pipeline de Contenido" a través del campo Categoría — cuando se genera contenido, el sistema busca en esta tabla los registros cuya Categoría coincida con la del contenido a generar.

Tabla 2: "Pipeline de Contenido" (núcleo del sistema — memoria y estado)

Campo	Tipo	Descripción
Idea Semilla	Single Line Text	El disparador temático que la IA va a desarrollar
Categoría	Single Select	Determina qué fragmentos del Manual de Marca se usan como contexto
Estado	Single Select	Generando → En revisión → Publicado / Rechazado
Contenido Generado	Long Text	Output de la IA
Aprobado	Checkbox	Marcado manualmente por el humano (punto HITL)
Notificado	Checkbox	Evita reenvíos duplicados de la notificación de aprobación

Campo	Tipo	Descripción
Canal	Single Select	Destino de publicación (ej. LinkedIn)
Ultima Modificacion	Last Modified Time	Campo trigger — dispara los 3 escenarios de Make
Created Time	Created Time	Timestamp de creación del registro

Relación: es la tabla que atraviesan los 3 escenarios — cada uno lee y escribe sobre los mismos registros, usando Estado y Aprobado como máquina de estados.

Tabla 3: "Log de Errores" (resiliencia)

Campo	Tipo	Descripción
Modulo	Single Line Text	Nombre del módulo de Make que falló
Fecha	Created Time	Se completa automáticamente
Mensaje	Long Text	El mensaje de error capturado ({{error.message}})

Relación: no está vinculada por Link Field a las otras tablas — se llena de forma independiente desde los Error Handlers de cualquiera de los 3 escenarios (registro de auditoría, no de negocio).

2. Esquemas JSON de transferencia entre integraciones

a) Output de "Search Records" (RAG) → input al Text Aggregator

```
{
  "id": "recXXXXXXXXXXXXXX",
  "fields": {
    "Categoría": "Servicios",
    "Contenido": "Ofrecemos consultoría personalizada de organización financiera para freelancers...",
    "Estado de Verificación": "Verificada"
  }
}
```

Puede devolver múltiples bundles (uno por cada fragmento que matchea la Categoría) — por eso es obligatorio el Text Aggregator antes de pasarlo a la IA.

b) Input al módulo "Gemini AI — Generate a Response"

```
{
  "model": "gemini-3.6-flash",
  "content": "Idea semilla: Tips de organización para freelancers\nCategoría: Servicios\n\nContexto validado de la marca: Ofrecemos consultoría personalizada..."
}
```

El campo content es 100% dinámico — se arma concatenando variables ({{3.Idea Semilla}}, {{3.Categoria}}, {{5.text}}), sin ningún valor hardcodedo.

c) Output de "Gemini AI" → input a "Update Record"

```
{
  "candidates": [
    {
      "content": {
        "parts": [
          { "text": "¿Sos freelancer y sentís que tu tiempo no rinde?..." }
        ]
      }
    }
  ]
}
```

Se mapea candidates[0].content.parts[0].text al campo Contenido Generado de Airtable.

d) Payload de "Slack — Send a Message" (notificación HITL)

```
{
  "channel": "aprobaciones-contenido",
  "text": "Nuevo contenido para revisar\n\nIdea Semilla: Tips de organización para\nfreelancers\nCategoría: Servicios\nCanal destino: LinkedIn\n\nContenido\ngenerado:\n¿Sos freelancer y sentís que tu tiempo no rinde?..."
}
```

e) Output de Airtable "Update Record" (registro final, tras aprobación)

```
{
  "id": "recECFm6VtGe07RrI",
  "fields": {
    "Idea Semilla": "Tips de organización para freelancers",
    "Categoría": "Servicios",
    "Estado": "Publicado",
    "Contenido Generado": "¿Sos freelancer y sentís que tu tiempo no rinde?...",
    "Aprobado": true,
    "Notificado": true,
    "Canal": "LinkedIn"
  }
}
```

Cuadro Comparativo de Costos

Proyecto: Pipeline de Generación y Publicación de Contenido con IA — Entrega Final

1. Motor de IA usado en la implementación

Por restricción presupuestaria, el motor de IA implementado en el flujo de Make es **Google Gemini (gemini-3.6-flash)**, que ofrece free tier sin tarjeta de crédito. La arquitectura es intercambiable: el módulo de IA es el único componente que cambiaría para usar Claude u OpenAI — el resto del flujo (Airtable, Slack, Router, Error Handlers) no se modifica.

El análisis de costos que sigue se hace igualmente con los modelos que pide la consigna (OpenAI y Anthropic), para demostrar el criterio de decisión que se aplicaría en un entorno de producción con presupuesto.

2. Justificación de modelo por tipo de tarea

Tarea	Modelo recomendado	Justificación
Generación de contenido con RAG (lectura densa del Manual de Marca, redacción creativa con tono de marca)	Claude (Sonnet)	Mejor comprensión de contexto largo y consistencia de tono en textos extensos; la tarea es de baja frecuencia (pocas piezas de contenido por día), por lo que el costo por token no es el factor crítico — la calidad sí.
Clasificación simple (ej. detectar la Categoría de una Idea Semilla, o triage de tickets)	GPT-4o-mini	Tarea de bajo razonamiento, alto volumen; el costo por token es 10-20x menor y el modelo alcanza igual precisión para clasificación de texto corto.
Procesamiento masivo/asincrónico (ej. regenerar todo el catálogo de contenido de una vez)	Batch API (Anthropic u OpenAI)	Descuento del 50% sobre la tarifa estándar a cambio de tiempo de respuesta no inmediato (hasta 24hs) — ideal para procesos que no requieren respuesta en tiempo real.

Prototipado / entornos sin presupuesto	Gemini (free tier)	Sin costo y sin tarjeta, con límite de 15 req/min — usado en esta entrega por restricción de presupuesto real.
--	---------------------------	--

3. Tabla de precios de referencia (por 1M de tokens, USD)

Modelo	Input estándar	Output estándar	Input con Batch (-50%)	Output con Batch (-50%)
Claude Sonnet	\$3.00	\$15.00	\$1.50	\$7.50
GPT-4o-mini	\$0.15	\$0.60	\$0.075	\$0.30
Gemini 3.6 Flash	\$1.50	\$7.00	\$0.75	\$3.75
Gemini free tier (usado en esta entrega)	\$0.00	\$0.00	—	—

4. Fórmula del descuento de Batch API

El descuento del 50% se aplica **de forma independiente** sobre la tarifa de input y sobre la de output — no es un descuento global sobre el costo total:

$\text{Costo_input_batch} = \text{tokens_input} \times (\text{tarifa_input_estandar} \times 0.5)$

$\text{Costo_output_batch} = \text{tokens_output} \times (\text{tarifa_output_estandar} \times 0.5)$

$\text{Costo_total_batch} = \text{Costo_input_batch} + \text{Costo_output_batch}$

Ejemplo aplicado (Claude Sonnet, generación de 1,000 piezas de contenido/mes, ~800 tokens input + ~400 tokens output cada una):

- Tokens totales: 800,000 input + 400,000 output por mes.
- **Costo estándar:** $(0.8 \times \$3.00) + (0.4 \times \$15.00) = \$2.40 + \$6.00 = \text{\$8.40/mes}$

- **Costo con Batch:** $(0.8 \times \$1.50) + (0.4 \times \$7.50) = \$1.20 + \$3.00 = \textbf{\$4.20/mes}$
- **Ahorro por Batch:** 50% exacto, ya que ambas tarifas bajan proporcionalmente.

5. Escenario intermedio: Prompt Caching sin Batch (aislando cada optimización)

Segun la devolución de la Pre-entrega 6, se agrega este escenario intermedio para separar el aporte de cada técnica:

Escenario	Descripción	Costo mensual (mismo volumen que el ejemplo anterior)
A — Sin optimización	Tarifa estándar, sin cache ni batch	\$8.40
B — Solo Prompt Caching	El bloque estático (contexto del Manual de Marca, ~500 de los 800 tokens de input) se cachea; solo se paga tarifa completa por el input dinámico (~300 tokens) + lectura de cache a 10% del costo de input estándar en el resto de las llamadas	$(0.3 \times \$3.00) + (0.5 \times \$0.30) + (0.4 \times \$15.00) = \$0.90 + \$0.15 + \$6.00 = \textbf{\$7.05}$
C — Caching + Batch combinados	Se aplica el 50% de Batch sobre las tarifas ya reducidas por caching	$(0.3 \times \$1.50) + (0.5 \times \$0.15) + (0.4 \times \$7.50) = \$0.45 + \$0.075 + \$3.00 = \textbf{\$3.53}$

Lectura del resultado: el Prompt Caching por sí solo aporta ~16% de ahorro (aísla el efecto de reusar el contexto estático), mientras que sumar Batch sobre el escenario ya cacheado casi duplica el ahorro total (~58% acumulado). Esto muestra que ambas optimizaciones son complementarias, no redundantes.

Condición técnica clave (de la devolución de PE6): para que `cache_control` active el hit de caché, el bloque estático (en este caso, el contexto del Manual de Marca) debe mantener **contenido y orden idénticos en cada ejecución** — cualquier variación en el texto o en el orden de los bloques invalida el cache y se cobra a tarifa completa.

6. Costo real de la implementación entregada

Como el flujo entregado usa Gemini en su free tier, el costo operativo actual es **\$0**, dentro del límite de 15 requests/minuto. Si el volumen de contenido creciera más allá de ese límite, la migración recomendada sería:

- 1. Gemini 3.6 Flash de pago (ya integrado, solo activar billing) para mantener el mismo costo bajo.
- 2. Migrar a Claude Sonnet con Prompt Caching + Batch API si se prioriza calidad de redacción sobre costo.

Seguridad y Resiliencia

Proyecto: Pipeline de Generación y Publicación de Contenido con IA — Entrega Final

1. Minimización de datos

El sistema solo envía a la IA los campos estrictamente necesarios para generar el contenido — nunca el registro completo de Airtable:

Campo enviado al prompt de Gemini	Motivo
Idea Semilla	Es el disparador temático, imprescindible
Categoría	Determina qué fragmentos del RAG se buscan
Contexto validado (texto del Manual de Marca ya filtrado)	Es la única fuente de verdad permitida por el system prompt

Campos que NUNCA se envían a la IA: Record ID, Ultima Modificacion, Created Time, Notificado, Aprobado, Canal. Estos son metadatos de control del flujo, no información de contenido, y no aportan nada al modelo — exponerlos sería una superficie de datos innecesaria.

Además, el System Prompt del módulo de IA restringe explícitamente el uso de conocimiento externo: *"Tu única fuente de verdad es el contexto que te paso a continuación... No uses conocimiento externo ni inventes datos sobre la empresa bajo ninguna circunstancia"*. Esto evita que el modelo alucine datos sensibles o inexactos de la marca.

La API key de Gemini se almacena únicamente dentro de la conexión de Make (no se expone en el prompt, en Airtable, ni en ningún campo visible), y no se comparte en capturas ni en el video demo.

2. Rutas de error (Error Handlers)

Se implementaron **4 Error Handlers**, todos con la directiva **Resume** (el flujo continúa en vez de cortarse), que registran el fallo en la tabla **Log de Errores** de Airtable:

Modulo protegido	Escenario	Campo registrado en caso de error
Gemini AI — Generate a Response	Escenario 1	Modulo="Gemini Prompt", Mensaje={{error.message}}
Airtable Update Record (Notificado)	Escenario 2	Modulo="Update Notificado", Mensaje={{error.message}}
Airtable Update Record (Publicado)	Escenario 3 — rama Aprobado	Modulo="Update Publicado", Mensaje={{error.message}}
Airtable Update Record (Rechazado)	Escenario 3 — rama Rechazado	Modulo="Update Rechazado", Mensaje={{error.message}}

Caso real capturado durante el test de estrés: la API de Gemini devolvió un error **[503] This model is currently experiencing high demand** durante una corrida real. El Error Handler permitió que el flujo no se rompiera; en un reintento inmediato, la operación se completó con normalidad. Esto valida en producción el comportamiento de resiliencia diseñado.

Camino infeliz (dato faltante): se probó también un registro con el campo **Idea Semilla** vacío. El sistema no rompió el flujo, pero generó contenido genérico de baja calidad (una pregunta de FAQ en vez de un post estructurado), ya que el modelo completó el vacío con el único dato disponible (la Categoría).

Mejora identificada: agregar un filtro adicional en el Escenario 1 que exija **Idea Semilla != vacío** antes de invocar a la IA, para evitar gastar créditos en generaciones sin sentido.

Filtro anti-bucle: cada Watch Records tiene un filtro de Estado específico (**Generando, En revisión AND Notificado=false, En revisión AND Aprobado=true**) que impide que un mismo registro sea reprocesado indefinidamente por el mismo escenario — una vez que el Estado cambia, el registro deja de matchear el filtro de origen.

3. Puntos de Human-in-the-loop (HITL)

El sistema tiene **un punto de validación humana obligatorio** antes de la acción crítica (publicar contenido en el canal de salida):

1. Cuando el contenido generado por la IA pasa a Estado "En revisión", el Escenario 2 notifica automáticamente al canal de Slack **#aprobaciones-contenido** con el texto completo generado.
2. El flujo **se detiene** en ese punto — ningún escenario publica nada hasta que un humano marca manualmente el checkbox **Aprobado** en Airtable.
3. Recién ahí, el Escenario 3 (disparado por ese cambio) enruta el contenido: si **Aprobado=true** lo publica (**Estado="Publicado"** + mensaje a **#contenido-publicado**); si no, lo marca **Rechazado** sin ninguna acción de salida.

Este diseño evita el "efecto metralleta" (publicar contenido sin supervisión) exigido por la consigna, y deja trazabilidad completa: cada pieza de contenido pasa por 3 estados verificables (Generando → En revisión → Publicado/Rechazado) antes de llegar al canal final.

Dashboard de Control

Proyecto: Pipeline de Generación y Publicación de Contenido con IA — Entrega Final

1. Enlace público

Shared View (Airtable — Kanban agrupado por Estado):

<https://airtable.com/appCtyPCA4QGzyXR5/shrBIC22XsubHnrfF>

Este enlace es distinto del link de lectura a la base completa — es una vista publicada específicamente como panel de control, de solo lectura, sin necesidad de cuenta de Airtable para verla.

2. Qué muestra el dashboard

La vista es un tablero Kanban de la tabla "Contenidos", agrupado por el campo Estado, con una columna por cada etapa del pipeline:

Columna (KPI)	Qué mide
Generando	Contenidos en proceso de generación por la IA (recén disparados, aún sin resultado)
En revisión	Contenidos generados que esperan la validación humana (HITL) — volumen de aprobaciones pendientes
Aprobado	Contenidos con el checkbox de aprobación marcado, a la espera de que el Escenario 3 los procese
Publicado	Contenidos que completaron el ciclo y salieron por el canal de Slack
Rechazado	Contenidos descartados por el revisor humano

Cada tarjeta muestra la Idea Semilla y la Categoría, permitiendo ver de un vistazo qué tipo de contenido predomina en cada etapa.

3. Tasa de errores

La base incluye una tabla adicional "Log de Errores", alimentada automáticamente por los 4 Error Handlers del sistema (ver Documento de Seguridad y Resiliencia). Cada fila registra: Módulo (dónde ocurrió el fallo), Fecha (automática) y Mensaje (el error capturado). La tasa de errores se calcula como:

Tasa de errores = (registros en Log de Errores) / (total de ejecuciones del pipeline)

Durante el test de estrés (5 corridas mínimas exigidas por la consigna, ejecutadas con más repeticiones para validar el fix de filtros), se registró 1 error real (fallo 503 de la API de Gemini por alta demanda),

correctamente capturado por el Error Handler sin interrumpir el flujo — tasa de error puntual, sin impacto en la integridad de los datos.

4. Por qué es un panel de control y no solo un link a la base

A diferencia del link de lectura a la base completa (que muestra todos los campos y registros sin curaduría), esta Shared View:

- Filtra y agrupa la información en KPIs accionables (conteo por estado).
- Es de solo lectura y no permite editar ni ver campos sensibles como API keys o configuraciones internas.
- Puede compartirse con un stakeholder no técnico (ej. un cliente) sin necesidad de explicarle la estructura completa de la base.